

재무탐정

공시 근거를 확인하는 재무 분석 AI

송준용 · 개인 프로젝트 · 2026.09 · AI 코딩 도구를 활용한 개발

긴 공시에서 설명 근거를 검색하고, 재무 수치의 계산·출처·기간을 함께 검증하는 웹 앱입니다. 단순 수치 조회에는 LLM을 호출하지 않고, 복합 질문에는 제한된 도구를 선택하는 Agent를 사용합니다.

구현한 사용자 흐름

기업 검색과 연간 수치 조회

미국 SEC·한국 OpenDART 연동. 매출·영업이익·영업현금흐름을 기간·통화·공시 링크와 표시하고, 비율은 Python으로 계산합니다.

공시 기반 답변

공시별 pgvector·BM25 하이브리드 검색 → 원문 구간을 인용한 답변 생성 → 인용 위치 대조 → 별도 LLM 근거 검토. 확인하지 못한 내용은 답변을 보류합니다.

도구를 연결한 분석

LangChain·LangGraph Agent가 재무 조회·계산·문단 검색·근거 추적 도구를 선택합니다. 계산은 조회된 관측값 ID만 받고 기간·통화·공시 범위를 검증합니다.

실제 실행 사례

“2025년 영업현금흐름에서 영업이익을 뺀 차이를 계산하고 원본 근거 경로를 보여줘”

선택 기업: 쿠팡 · 기준 공시: 2025년 10-K · 통화: USD

1,773,000,000 - 473,000,000 = 1,300,000,000 USD

실제 도구 순서: get_financials → calculate_metrics → trace_evidence

관측 실행: 약 6.7초 · 예상 API 비용 \$0.002415 · 2026-09-21 평가 기록

해당 공시는 미리 색인된 상태였으며, 한 실행의 관측값입니다. 최초 수집·임베딩 시간과 비용, 서버·DB 비용은 포함하지 않습니다.

쿠팡 2025년 10-K 원문

아키텍처

React / Vite → FastAPI → 질문 라우팅 → 수치 조회 또는 RAG / Agent → 결과 검증

PostgreSQL: 공시·문단·512차원 임베딩·실행·비용·캐시의 기준 기록

Neo4j: 기업·공시·기간·관측값·지표·근거의 관계 탐색 PoC

Docker Compose: PostgreSQL/pgvector·Neo4j 로컬 실행

실패를 발견하고 개선한 과정

삼성전자 반도체 사업 요약에서 회사 전체 투자·실적을 사업부 설명에 섞는 오류가 발생했습니다. 문단 검색은 성공했지만 별도 검토 단계가 근거 범위 불일치를 탐지해 답변을 보류했습니다.

- 프롬프트 개선: 질문한 사업 범위만 요약하고 회사 전체 수치·현황으로 답변을 확대하지 않도록 지시
- 응답 계약 개선: 프롬프트 지시만으로 주장 수가 제한되지 않아, Structured Output 스키마와 Pydantic의 개수·길이 제약을 일치시킴
- 검증 범위 개선: 검토되지 않는 자유 서술 필드가 추가 사실을 전달하지 않도록 서버가 검색 범위 안내를 작성하고 회귀 테스트 추가

검증 결과와 해석 범위

자동화 검증	pytest 68개 통과. 합성 모델 응답과 실제 PostgreSQL·Neo4j로 기간·출처·계산·실행 한도·비용·사용자 격리 검증
실제 모델 평가	고정 개발 사례 8개 기준 통과. AI 실행 5개와 범위 차단 3개. 일반 정확도나 독립 평가의 환각률을 의미하지 않음
개선 기록	최초 7/8 → 프롬프트·응답 스키마 개선 후 8/8. 질문을 보며 개선한 개발 집합임을 평가 보고서에 명시
비용 관측	해당 최종 8개 실행 합계 \$0.032146. 이전 실패, 문서 색인, 호스팅·세금 제외

설계 판단과 한계

- Ontology는 기업·공시·기간·통화·수치·근거의 의미와 관계를 명시합니다. 원본 수치와 서술문을 구분해 단순 관련성을 인과관계로 저장하지 않습니다.
- Neo4j는 관계 탐색 PoC입니다. 현재 짧은 근거 경로는 SQL JOIN으로도 구현 가능하며, 그래프 DB가 필수라고 주장하지 않습니다. 복잡한 의존성 질의와 운영 비용으로 채택 가치를 추가 평가해야 합니다.
- 지원 범위는 선택 기업의 연간 공시·3개 주요 재무 지표입니다. 모든 기업의 전용 계정·문서 형식, 분기·기업 간 비교·장기 대화 기억은 아직 지원하지 않습니다.
- 로컬 MVP를 검증했습니다. 대규모 사용자 부하, 공개 운영 배포, 작업 큐·장애 후 실행 재개·백업 복구 검증은 후속 과제입니다.

검토 자료

GitHub · [finance-detective](#)

소스는 비공개이며 요청 시 열람을 제공합니다. 저장소의 README, docs/architecture.md, docs/agent-graph-guide.md, evals/agent-report.json에 코드 구조와 검증 기록을 정리했습니다.

기준 코드: fa6ccf6 · 핵심 기능 검증: 2026-09-21 · 자료 작성: 2026-09-22